

Retrivo

Information Retrieval System

A full-pipeline IR system — Boolean, TF-IDF, Evaluation, Statistics

Under Supervision
of

Dr. Ebtissam AbdelHakam

TA. Marco Edwar

Project Team

1. Mohamed Sayed Saad Mojawer
2. Ammar Yasser AbdEllatif AbdElAzim
3. Beshoy Farouk Gaber
4. Hussien Mohamed Haggag
5. Michael Hany Kamal

Live Deployment: <https://retrivoapp.dpdns.org>

1. Project Overview

Retrivo is a complete, web-based Information Retrieval system built as a course project for the Information Retrieval course at Minia University. The system demonstrates the full IR pipeline from raw text preprocessing through indexing, ranked retrieval, and quantitative evaluation. It is deployed at <https://retrivoapp.dpdns.org>.

The system operates on a corpus of 100 Wikipedia articles and exposes three retrieval models through a responsive web interface. All preprocessing is consistent between documents and queries, and all six course lab sections are directly reflected in the implementation.

Property	Details
Corpus	100 Wikipedia articles — 5 domains, 20 per domain
Retrieval Models	Boolean TDM, Boolean Inverted Index, TF-IDF + Cosine Similarity
Boolean Operators	AND, OR, NOT — supported in all Boolean retrieval methods
Preprocessing	Tokenization + Stop word removal + Stemming (Porter/Snowball/Lancaster) or WordNet Lemmatization
Evaluation	Precision, Recall, F1, Average Precision, Mean Average Precision (MAP)
Deployment	https://retrivoapp.dpdns.org (HTTPS)

2. Dataset — Corpus Preparation

2.1 Source and Rationale

The 20 Newsgroups dataset was initially evaluated but rejected because its documents are informal discussion threads whose vocabulary does not match domain terminology. Wikipedia article summaries were chosen instead for their authoritative, keyword-rich content that reliably matches user queries.

2.2 Domain Topics

Domain	Articles	Representative Topics
Medicine	20	Surgery, Pharmacology, Cardiology, Neurology, Oncology, Vaccine, Cancer, Diabetes
Space	20	Astronomy, NASA, Solar System, Black Hole, Mars, Rocket, Astronaut, Hubble
Computer Science	20	AI, Machine Learning, Deep Learning, NLP, Cybersecurity, Information Retrieval

Sports	20	Football, Basketball, Tennis, Cricket, Olympic Games, Ice Hockey, Baseball
Politics	20	Democracy, United Nations, Human Rights, Election, Parliament, Revolution

3. Text Preprocessing Pipeline

Every document in the corpus and every incoming user query pass through the same three-step preprocessing pipeline. This consistency between document indexing and query processing is essential — without it, vocabulary mismatches cause retrieval failures even when relevant documents exist.

3.1 Tokenization

Raw text is converted to lowercase and split into word tokens using NLTK's `word_tokenize`. Punctuation marks and numeric tokens are discarded. Contractions are handled correctly — "I'm" becomes ["i", "m"] rather than a single unknown token.

3.2 Stop Word Removal

Common English words — "the", "is", "and", "of" and 176 others — are removed using NLTK's English stop word list. These words carry no discriminating power between documents.

3.3 Stemming and Lemmatization

The system provides four interchangeable normalisation strategies, selectable by the user:

Algorithm	Type	Characteristics
Porter	Stemmer	Moderate — removes common suffixes. Fast and widely used.
Snowball	Stemmer	Slightly more accurate than Porter. Supports multiple languages.
Lancaster	Stemmer	Aggressive — cuts words very short. Fastest but may over-stem.
WordNet Lemmatizer	Lemmatizer	Uses POS tagging to return valid dictionary root forms. Most linguistically accurate.

Every search also shows a live Preprocessing Trace panel — the user can see their query pass step-by-step through tokenization, stop word removal, and processing, plus a side-by-side comparison of all four algorithms on the same query.

4. Boolean Retrieval — AND, OR, NOT

Both Boolean retrieval methods (Term-Document Matrix and Inverted Index) now support a full Boolean query parser that recognises the operators AND, OR, and NOT in any combination within a query string.

4.1 Query Syntax

Example Query	Operator	Meaning
space AND nasa	AND (default)	Documents containing both terms
hockey OR baseball	OR	Documents containing either term
space NOT military	NOT	Space documents excluding military
space AND nasa NOT military	Mixed	Chained operators evaluated left to right

4.2 Term-Document Matrix

The TDM is a binary grid where rows are terms and columns are documents. A cell is 1 if the term appears in that document. Boolean queries execute set operations across the relevant matrix rows. AND is intersection, OR is union, NOT is set difference.

4.3 Inverted Index

The Inverted Index maps each term to its postings list — the list of documents containing it. Boolean queries intersect and union these postings lists directly, which is more efficient for large corpora since most terms appear in only a small fraction of documents.

5. TF-IDF with Cosine Similarity

TF-IDF (Term Frequency – Inverse Document Frequency) is a ranked retrieval model that assigns each term in each document a weight reflecting its importance relative to the corpus.

- Term Frequency (TF): how often a term appears in a document, normalised by document length.

- Inverse Document Frequency (IDF): $\log(N / df)$ where N is the total number of documents and df is the number of documents containing the term. Rare terms get higher IDF.
- TF-IDF score = $TF \times IDF$. Each document is a vector of these scores across the vocabulary.
- Cosine Similarity measures the angle between the query vector and each document vector. Results are ranked in descending order of cosine similarity.

Critical implementation note: The same preprocessing pipeline (stemming or lemmatization) is applied to both the document corpus at index time and to every incoming query at search time. Without this, a query for "medicine" would never match documents indexed under the stem "medicin".

6. Vocabulary and IDF Statistics

A dedicated Statistics page exposes corpus-level insights derived from the fitted TF-IDF model. This section directly visualises the IDF concept from Section 5 of the course.

- Corpus size — total document count and total unique term count after preprocessing.
- Most Specific Terms (Highest IDF) — terms that appear in very few documents and therefore carry the most discriminating power. These are the terms most characteristic of a single domain.
- Most Common Terms (Lowest IDF) — terms that appear across many documents, indicating low discrimination. Useful for understanding what vocabulary is shared across domains.
- Highest Average TF-IDF — terms with the highest mean TF-IDF score across all documents, identifying the most globally important terms in the corpus.

All three lists are displayed with proportional bar charts and exact scores, and can be recalculated instantly by switching between Porter, Snowball, and Lancaster stemmers to observe how different preprocessing affects the vocabulary.

7. Evaluation Metrics

7.1 Per-Query Evaluation

When the user provides a list of known-relevant document filenames in the evaluation field, the system computes four metrics against the retrieved results:

Metric	What It Measures	Perfect Score
Precision	Fraction of retrieved docs that are relevant	1.0 (no irrelevant docs returned)
Recall	Fraction of relevant docs that were retrieved	1.0 (no relevant docs missed)
F1-Score	Harmonic mean of Precision and Recall	1.0 (perfect precision and recall)

Average Precision	Ranking-aware — rewards placing relevant docs early	1.0 (all relevant docs ranked first)
-------------------	---	--------------------------------------

7.2 Mean Average Precision (MAP) — Batch Evaluation

The dedicated MAP Evaluation page runs an automated batch evaluation across five predefined queries — one per domain — with automatically determined relevant document sets (all documents from the matching domain).

For each query, the system retrieves the top-20 TF-IDF results and computes Precision, Recall, F1, and Average Precision. The MAP score is the mean of all five AP values and provides a single-number summary of overall system quality.

The MAP evaluation can be re-run with any of the three stemmers to directly compare how the choice of preprocessing algorithm affects system performance — a direct demonstration of the evaluation concepts from Section 6 of the course.

8. Document Preview with TF-IDF Weights

Clicking the Preview button on any search result opens a modal panel showing the first 1,200 characters of that document alongside the top 10 TF-IDF weighted terms for that specific document. Each term is shown with its exact TF-IDF score and a proportional bar indicating its weight relative to the highest-scoring term.

This feature directly visualises the TF-IDF DataFrame concept from Section 5 of the course, showing which terms most strongly characterise each individual document.

9. System Architecture

9.1 `ir_engine.py`

All IR logic is isolated in this module. It loads all documents at startup, applies preprocessing, and builds all indexes in memory. Key functions:

- `preprocess(text, stemmer, use_lemma, trace)` — unified preprocessing for documents and queries, returns tokens or a step-by-step trace dict.
- `build_indexes()` — constructs the Term-Document Matrix and Inverted Index.
- `build_tfidf()` — fits a TF-IDF vectorizer on the preprocessed corpus.
- `boolean_query_td/inverted()` — Boolean AND/OR/NOT query execution.
- `tfidf_query()` — cosine similarity ranking.

- `get_doc_tfidf_weights()` — top-N TF-IDF terms for a specific document.
- `get_vocabulary_stats()` — corpus-level IDF and TF-IDF statistics.
- `mean_average_precision()` — MAP computation across multiple queries.

9.2 app.py

Endpoint	Purpose
GET /	Serves the main web interface
POST /search	Runs query — returns results, trace, and evaluation metrics
POST /preprocess	Returns pipeline trace and all-stemmer comparison for a query
GET /document/<name>	Document preview text + top TF-IDF term weights
GET /stats	Vocabulary and IDF statistics for the corpus
POST /evaluate_map	Batch MAP evaluation across 5 predefined domain queries

9.3 Deployment

The system is deployed at <https://retrivoapp.dpdns.org>. Nginx acts as a reverse proxy with HTTPS termination. Gunicorn serves the Flask application with two worker processes. The application is managed by systemd for automatic startup and restart on failure.

10. Implementation Steps Summary

1. Studied all six course lab sections covering tokenization, stop words, stemming, lemmatization, inverted index, TDM, TF-IDF, cosine similarity, and evaluation metrics.
2. Designed a two-module architecture separating IR logic (`ir_engine.py`) from the web layer (`app.py`).
3. Implemented a unified preprocessing pipeline supporting Porter, Snowball, and Lancaster stemmers plus WordNet lemmatization, applied consistently to both documents and queries.
4. Built a live preprocessing trace showing each query step-by-step through the pipeline, with a side-by-side stemmer comparison.
5. Implemented the Term-Document Matrix and Inverted Index with a full Boolean query parser supporting AND, OR, and NOT operators.
6. Implemented TF-IDF vectorization with the critical fix of preprocessing both documents and queries before vectorization, resolving vocabulary mismatches.
7. Added cosine similarity ranking with top-10 result retrieval and per-document TF-IDF weight visualization.
8. Implemented all evaluation metrics: Precision, Recall, F1, Average Precision, and MAP.
9. Built the MAP batch evaluation page with five predefined domain queries and automatic relevant document detection.

10. Built the Vocabulary & IDF Statistics page showing most specific, most common, and most discriminative terms with proportional bar charts.
11. Evaluated and rejected the 20 Newsgroups dataset; adopted Wikipedia article summaries via the wikipedia-api library.
12. Deployed the system as a secured, production-ready web application at <https://retrivoapp.dpdns.org>.

11. Conclusion

Retrivo demonstrates the complete pipeline of a functional Information Retrieval system, from raw text to ranked and evaluated results. The project covers all three principal IR techniques from the course: Boolean retrieval through a Term-Document Matrix, Boolean retrieval through an Inverted Index (both now supporting AND/OR/NOT operators), and ranked retrieval through TF-IDF with cosine similarity.

The extended features — stemmer and lemmatizer selection, preprocessing trace, vocabulary statistics, document TF-IDF visualization, and MAP batch evaluation — bridge the gap between theoretical course concepts and a working system, making every concept from Sections 1 through 6 directly observable through the interface.

A key lesson from the implementation is that preprocessing consistency between indexing and querying is not optional — it is the foundation on which all retrieval quality depends. The stemmer comparison feature makes this visible by showing how different algorithms produce different stems from the same query, directly affecting which documents are matched.

The system is live at <https://retrivoapp.dpdns.org>.